



Recursive vs Iterative Algorithms

Brief Overview

This note covering Recursive and Iterative Computations was created from a [6-page PDF](#). It covers **Recursive definition** and **Iterative definition**, factorial implementations, stack frame mechanics, algorithm comparison metrics, recursive data structures, and Fibonacci recursion.

Key Points

- Factorial computation: recursive vs iterative Java code snippets.
 - Stack frames: how calls are pushed and popped on the call stack.
 - Algorithm comparison: step count vs memory usage.
 - Recursive data structures: lists, binary search trees, and Fibonacci recurrence.
-



Factorial: Recursive vs Iterative

Recursive definition – a function that calls itself with a simpler argument until reaching a *base case*.

Iterative definition – a function that uses loops to compute the result without self-calls.

Recursive implementation (Java)

```
int factorial(int n) {
    if (n == 1 || n == 0)           // base case
        return 1;
    else
        return n * factorial(n-1); // recursive case
}
System.out.print(factorial(5));
```

Iterative implementation (Java)

```
int factorial(int n) {
    int prod = 1;
    for (int i = 1; i <= n; i++)    // note: loop runs through *n* inclusive
        prod = prod * i;
```

```
    return prod;
}
```

Comparison Table

Aspect	Recursive	Iterative
Number of steps	Grows with depth of recursion; each call adds constant work.	Linear in n (single loop).
Extra memory	Uses a stack frame per call $\rightarrow O(n)$ space.	Constant extra space $\rightarrow O(1)$.
Readability	Mirrors mathematical definition; concise.	May be more verbose but avoids call-overhead.



Stack Frames

A **stack frame** stores a method's *parameters*, *return value*, and *return address* while the method is active.

- When a method is invoked, a new frame is **pushed** onto the call stack.
- Upon returning, the top frame is **popped** and discarded.



Algorithm Comparison

1. **Step count** – count of primitive operations (e.g., multiplications, assignments).
2. **Extra memory** – additional space beyond input size (e.g., stack frames for recursion).

These metrics help decide between recursive and iterative solutions.

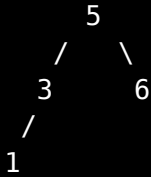


Recursive Data Structures

- **Lists** can be expressed recursively: a *head* element followed by a *sub-list* (the tail).
Example: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 10$.
- **Binary Search Tree (BST)** properties:

Each node has 0, 1, or 2 children. Left child < node value; right child > node value; duplicates are not allowed.

Example tree (root = 5):



Fibonacci Sequence

- Sequence: 1, 1, 2, 3, 5, 8, 13, 21, ...
- **Recurrence relation**

$$fib(0) = 1, fib(1) = 1$$

$$fib(n) = fib(n - 1) + fib(n - 2) \quad \text{for } n \geq 2$$

- **Counting steps** for the naïve recursive algorithm is exponential:

Approximately 2^n function calls.

- **Memory usage** grows with recursion depth (up to n stack frames).
-